

Scanning and Enumeration

1. Where This Fits in the Penetration Testing Methodology

Recall the standard penetration testing lifecycle:

1. Pre-Engagement
2. Reconnaissance / Information Gathering
3. **Scanning & Enumeration** ← *you are here*
4. Vulnerability Analysis
5. Exploitation
6. Post-Exploitation
7. Reporting

If reconnaissance answers the question *"What does this organization look like from the outside, and what exists?"*, scanning answers the much more technical question: *"Of everything we found, what is actually alive, reachable, and running — and how?"*

This is the phase where a tester transitions from **passive** information gathering into **active** engagement with the target's systems. From this point forward, the tester is directly sending packets to target infrastructure, which means activity can be logged, detected, and — if out of scope — legally and professionally consequential. This is why scanning must always occur strictly within the boundaries defined in the Rules of Engagement.

2. What Is Scanning?

Scanning is the process of actively probing a target network or system to identify:

- Which hosts are **live** (up and reachable)
- Which **ports** are open, closed, or filtered on those hosts
- Which **services** are running behind those ports
- What **operating system** and software versions are in use
- What the general **network topology** and **security posture** look like

Technically, scanning works by sending specially crafted packets to a target and analyzing the responses (or lack thereof) to infer the state of the system. This is fundamentally a game of **structured trial and observation** — the scanner sends a stimulus, and the target's response (or silence) reveals information.

2.1 Scanning vs. Footprinting — The Key Distinction

Footprinting (Recon)

Scanning

Footprinting (Recon)	Scanning
Mostly passive	Mostly active
Broad, organizational, OSINT-driven	Narrow, technical, network-driven
Rarely detectable	Can be detected by IDS/IPS/firewalls
Produces: domains, employees, general infrastructure	Produces: live hosts, open ports, service versions, OS fingerprints

2.2 Scanning vs. Enumeration

These two terms are often used together but are technically distinct:

- **Scanning** identifies *what exists* — live hosts, open ports, running services (surface-level detection)
- **Enumeration** goes one level deeper — actively extracting detailed information from identified services, such as usernames, shares, group memberships, DNS records, SNMP data, and application-specific details

Think of scanning as "**knocking on every door to see which ones are unlocked,**" and enumeration as "**walking through the unlocked doors to see what's inside the room.**"

3. Why Do We Scan?

Understanding the *purpose* behind scanning is just as important as knowing the mechanics.

3.1 From an Offensive (Red Team / Pentester) Perspective

- **Attack surface mapping** — build a precise inventory of live systems and exposed services to target
- **Vulnerability targeting** — knowing the exact service and version (e.g., "OpenSSH 7.2p2") allows testers to search for known CVEs specific to that version
- **Prioritization** — not all systems are equal; scanning helps identify high-value targets (domain controllers, database servers, exposed admin panels)
- **Validating recon findings** — confirming that hosts discovered passively are actually live and responsive
- **Building an exploitation plan** — scanning results directly feed into which exploits, payloads, and techniques will be attempted next

3.2 From a Defensive (Blue Team) Perspective

Scanning is not exclusively an attacker's tool — defenders use the exact same techniques for:

- **Asset inventory management** — knowing what's actually running on your network (you can't protect what you don't know exists)

- **Attack surface reduction** — identifying and closing unnecessary open ports/services
- **Vulnerability management programs** — regular scanning to catch outdated, vulnerable software before attackers do
- **Compliance audits** — many standards (PCI-DSS, NIST) mandate regular network scanning
- **Detecting rogue devices** — unauthorized systems connected to the network (shadow IT)

Professional Insight: As a cybersecurity student, understand that the *tool* (e.g., Nmap) is neutral — what determines whether its use is legal and ethical is **authorization and intent**, not the tool itself.

4. Network Scanning

Network scanning refers specifically to the process of discovering active devices on a network and mapping the network's structure. It is typically the very first activity in the technical phase, answering the foundational question: *"Which IP addresses in this range actually have something running on them?"*

4.1 Core Objectives of Network Scanning

1. **Host Discovery** — determine which IP addresses are live
2. **Port Scanning** — determine which ports are open on live hosts
3. **Service/Version Detection** — determine what software is answering on those ports
4. **OS Fingerprinting** — determine the underlying operating system
5. **Topology Mapping** — understand how systems relate to each other on the network (routers, subnets, firewalls)

4.2 Types of Network Scans (By Purpose)

Scan Type	Purpose
Host Discovery Scan	Find which hosts are up (ping sweep)
Port Scan	Find open/closed/filtered ports on a host
Service/Version Scan	Identify software and version behind a port
OS Detection Scan	Fingerprint the operating system
Vulnerability Scan	Cross-reference findings against known CVEs (often via dedicated tools like Nessus/OpenVAS, layered on top of scanning results)

4.3 Underlying Protocols Used in Scanning

Scanning relies heavily on manipulating standard networking protocols and observing how targets respond:

- **TCP (Transmission Control Protocol)** — connection-oriented; the 3-way handshake (SYN → SYN/ACK → ACK) is the basis for most TCP scan techniques
- **UDP (User Datagram Protocol)** — connectionless; harder to scan reliably because there's no handshake, and closed ports often don't respond at all
- **ICMP (Internet Control Message Protocol)** — used for ping sweeps and network diagnostics (echo request/reply)
- **ARP (Address Resolution Protocol)** — used for host discovery on local (same-subnet) networks, since ARP requests are almost always answered even when ICMP is blocked

5. Nmap — The Industry-Standard Scanning Tool

Nmap ("Network Mapper") is the most widely used open-source tool for network discovery and security auditing. Written by Gordon Lyon ("Fyodor"), it has been the de facto standard scanning tool in both offensive and defensive security for over two decades.

5.1 Why Nmap Dominates the Industry

- Extremely flexible — supports dozens of scan types and thousands of option combinations
- Cross-platform (Linux, Windows, macOS)
- Extensible via the **Nmap Scripting Engine (NSE)** — covered in Section 9
- Free, open-source, and extensively documented
- Has both a CLI version and a GUI front-end (**Zenmap**)

5.2 Basic Nmap Syntax

```
nmap [scan type] [options] [target]
```

5.3 Common Nmap Scan Types

Flag	Scan Type	Description
-sS	TCP SYN Scan ("Stealth Scan")	Sends SYN, analyzes response, never completes handshake — fast and relatively stealthy
-sT	TCP Connect Scan	Completes the full 3-way handshake — more detectable, but doesn't require raw socket privileges
-sU	UDP Scan	Scans UDP ports — slower and less reliable due to UDP's connectionless nature
-sA	ACK Scan	Used to map firewall rulesets rather than find open ports
-sN / -sF / -sX	Null, FIN, Xmas Scans	Manipulate unusual TCP flag combinations to evade certain basic firewalls/IDS

Flag	Scan Type	Description
-sV	Service/Version Detection	Identifies software and version running on open ports
-O	OS Detection	Fingerprints the target's operating system
-A	Aggressive Scan	Combines OS detection, version detection, script scanning, and traceroute
-p-	All Ports	Scans all 65535 TCP ports (default Nmap only scans the top 1000)
-Pn	Skip Host Discovery	Treats all hosts as online, skipping the ping step (useful when ICMP is blocked)

5.4 Example Commands

```
# Basic scan of a single host
nmap 192.168.1.10
```

```
# Scan a subnet
nmap 192.168.1.0/24
```

```
# Stealthy SYN scan with service/version detection on all ports
nmap -sS -sV -p- 192.168.1.10
```

```
# Aggressive scan with OS detection, scripts, and traceroute
nmap -A 192.168.1.10
```

```
# Scan while evading host discovery (target blocks ping)
nmap -Pn -sS 192.168.1.10
```

5.5 Interpreting Nmap Output

Nmap classifies each scanned port into one of these states:

- **Open** — an application is actively accepting connections on this port
- **Closed** — the port is reachable, but no application is listening
- **Filtered** — Nmap cannot determine the state because a firewall/filter is blocking probes
- **Unfiltered** — the port is accessible, but Nmap cannot determine if it's open or closed (mainly seen with ACK scans)
- **Open|Filtered** — Nmap cannot determine whether the port is open or filtered (common with UDP scans)

6. Host Detection (Host Discovery)

Before scanning ports, a tester typically first determines **which hosts in a given range are actually alive** — there's no point scanning 65,535 ports on an IP address that isn't even in use.

6.1 Common Host Discovery Techniques

- **ICMP Echo Request (Ping Sweep)** — sends ICMP echo requests across a range; a reply indicates a live host (`-sn` in Nmap performs a "ping scan" without port scanning)
- **ARP Scanning** — on local networks, ARP requests are used because they operate at Layer 2 and are rarely blocked, making them very reliable for local host discovery
- **TCP SYN/ACK Ping** — sends TCP packets to commonly open ports (e.g., 80, 443) to elicit a response even if ICMP is blocked
- **UDP Ping** — sends UDP packets to typically closed ports; a resulting ICMP "port unreachable" message confirms the host is alive

6.2 Nmap Host Discovery Examples

```
# Ping sweep only (no port scan)
nmap -sn 192.168.1.0/24

# ARP scan (fast, reliable on local subnet)
nmap -PR 192.168.1.0/24

# TCP SYN ping to specific ports
nmap -PS22,80,443 192.168.1.0/24
```

6.3 Why Host Discovery Can Fail

Many organizations block ICMP at the perimeter firewall specifically to prevent easy ping sweeps. A skilled tester must know how to pivot to alternative discovery methods (TCP/UDP ping, ARP on local segments) rather than assuming a host is down just because it doesn't respond to ICMP.

7. Port Detection (Port Scanning)

Once live hosts are identified, the next step is determining **which of the 65,535 possible TCP/UDP ports are open** on each host.

7.1 The TCP Three-Way Handshake (Foundation of Most Scans)

Understanding TCP scanning requires understanding the handshake it manipulates:

1. Client sends **SYN**
2. Server responds with **SYN/ACK** (if port is open) or **RST** (if port is closed)
3. Client sends **ACK** to complete the connection (only in a full connect scan)

7.2 Major Port Scanning Techniques

TCP SYN Scan ("Half-Open" Scan) Sends a SYN packet; if SYN/ACK is received, the port is open, but the tester sends an **RST** instead of completing the handshake. This is faster and historically less likely to be logged by applications (since the full

connection is never established at the application layer), though modern IDS/IPS solutions detect this pattern easily.

TCP Connect Scan Completes the full three-way handshake. Used when the tester lacks raw socket privileges (e.g., non-root/non-administrator access), but it is more easily logged since it creates a full application-layer connection.

UDP Scan Sends a UDP packet and waits:

- No response → likely **open** or **filtered**
- ICMP "port unreachable" → **closed**
- A UDP response → **open** UDP scanning is inherently slower and less reliable than TCP scanning because many systems rate-limit or silently drop unsolicited UDP/ICMP traffic.

Stealth Scans: NULL, FIN, and Xmas These manipulate unusual TCP flag combinations that violate the RFC-expected handshake behavior:

- **NULL scan** — no flags set at all
- **FIN scan** — only the FIN flag set
- **Xmas scan** — FIN, PSH, and URG flags all set ("lit up like a Christmas tree")

On RFC-compliant systems, closed ports respond with an RST, while open ports send no response at all — allowing inference of port state. These scans can sometimes evade simple stateless firewalls but are unreliable against modern stateful systems (and don't work reliably against Windows targets, which don't follow the RFC behavior consistently).

7.3 Port States Recap

State	Meaning
Open	Actively listening, accepting connections
Closed	Reachable, but nothing is listening
Filtered	A firewall/ACL is blocking probes — state unknown
Open/Filtered	Ambiguous result, common in UDP scanning

8. OS Detection (OS Fingerprinting)

OS fingerprinting is the process of determining a target's operating system by analyzing subtle, distinguishing characteristics in how it implements networking protocols.

8.1 Why OS Detection Matters

- Different operating systems have different **known vulnerabilities** and patch cycles
- Exploit selection is often OS-specific (a Windows exploit won't work on Linux)
- Helps testers understand what **post-exploitation techniques** will apply

- Assists in accurately mapping the network (e.g., identifying network appliances vs. servers vs. workstations)

8.2 Passive vs. Active OS Fingerprinting

- **Active fingerprinting** — the tester sends crafted probe packets and analyzes the response characteristics directly (this is what Nmap's `-O` flag does)
- **Passive fingerprinting** — the tester silently observes traffic already flowing (e.g., via `poft`) without sending anything, inferring OS from natural packet characteristics — stealthier but requires existing traffic to observe

8.3 How Active OS Fingerprinting Works (Technical Basis)

Every OS's TCP/IP stack implementation has subtle quirks — a "fingerprint" — based on:

- **Initial TTL (Time to Live) values** — e.g., Linux often defaults to 64, Windows to 128
- **TCP Window Size** — the initial window size advertised varies by OS and version
- **TCP Options ordering and support** — different OSes support/order options (like MSS, window scaling, SACK, timestamps) differently
- **Response to malformed/unusual packets** — different stacks handle malformed packets in observably different ways
- **IP ID sequence generation patterns**

Nmap compares observed responses against a large signature database (`nmap-os-db`) to make a probabilistic match, often returning a percentage confidence for each guess.

8.4 Example

```
nmap -O 192.168.1.10
```

Typical output includes the best-guess OS along with a confidence percentage, and sometimes multiple candidate matches when the fingerprint is ambiguous.

9. Banner Grabbing

Banner grabbing is a technique used to gather information directly from the **service banners** that many applications broadcast upon connection — text that often reveals the software name, version number, and sometimes even the underlying OS.

9.1 Why Banners Are So Valuable

A banner like:

```
220 ProFTPD 1.3.5 Server ready
```

immediately tells a tester the exact FTP server software and version — allowing a quick lookup for known CVEs affecting that specific version (e.g., ProFTPD 1.3.5 has

publicly known backdoor vulnerabilities). This is often one of the fastest paths from scanning straight to a working exploit.

9.2 Manual Banner Grabbing Techniques

```
# Using Netcat
nc -nv 192.168.1.10 21

# Using Telnet
telnet 192.168.1.10 80
GET / HTTP/1.1
Host: target.com

# Using cURL for HTTP headers
curl -I http://target.com
```

9.3 Automated Banner Grabbing

Nmap's `-sV` flag automates banner/version detection across all discovered open ports:

```
nmap -sV 192.168.1.10
```

9.4 Defensive Countermeasures Against Banner Grabbing

- **Banner suppression/modification** — configuring services to hide or alter version strings
- **Generic error messages** — avoiding overly descriptive server responses
- **Regular patching** — reduces the risk even if version info is exposed, since there's nothing exploitable to find
- Note: security through obscurity (hiding banners) is a **supplementary** control, not a substitute for actually patching vulnerable software

10. NSE — The Nmap Scripting Engine

10.1 What Is NSE?

The **Nmap Scripting Engine (NSE)** is one of Nmap's most powerful features — it allows users to write and run **Lua scripts** that automate a huge range of tasks beyond basic port scanning: vulnerability detection, deeper enumeration, exploitation of certain flaws, backdoor detection, and more.

10.2 NSE Script Categories

NSE scripts are organized into categories that describe their purpose and risk level:

Category	Purpose
auth	Bypass or test authentication mechanisms
broadcast	Discover hosts via broadcast/multicast requests

Category	Purpose
brute	Perform brute-force credential attacks
default	Scripts run automatically with -sC or -A (safe, informative)
discovery	Deeper enumeration of accessible services (e.g., SMB shares, SNMP data)
dos	Test for denial-of-service conditions (use with extreme caution — can crash services)
exploit	Actively exploit a known vulnerability
safe	Scripts unlikely to crash or disrupt the target
vuln	Check for specific known vulnerabilities

10.3 Using NSE

```
# Run default safe scripts
nmap -sC 192.168.1.10

# Run a specific script category
nmap --script vuln 192.168.1.10

# Run a specific named script
nmap --script http-title 192.168.1.10

# Combine with aggressive scanning
nmap -A 192.168.1.10
```

10.4 Practical NSE Examples

- `smb-os-discovery` — extracts OS and workgroup/domain info via SMB
- `http-title` — grabs the `<title>` tag of a web page
- `ftp-anon` — checks whether anonymous FTP login is permitted
- `ssl-cert` — extracts SSL/TLS certificate details, including Subject Alternative Names
- `vulners` — cross-references detected service versions against known CVE databases

Caution for Students: Scripts in the exploit, dos, and brute categories can cause real disruption (crashed services, locked accounts) if run carelessly. Always confirm scope and authorization explicitly covers this level of intrusiveness before using them.

11. Firewall and IDS/IPS Evasion Techniques

As scans become more sophisticated, so do the defenses designed to detect and block them. Understanding evasion techniques is essential both for red teamers simulating a determined adversary and for blue teamers who need to understand what their defenses must actually catch.

11.1 Why Evasion Matters

Modern networks are protected by:

- **Firewalls** — filter traffic based on rules (port, protocol, source/destination)
- **IDS (Intrusion Detection Systems)** — passively monitor and alert on suspicious traffic patterns
- **IPS (Intrusion Prevention Systems)** — actively block traffic matching known-malicious patterns

A "loud" scan (e.g., a full-speed `-p- -sV` sweep of every port) is trivially detected by any modern IDS. Evasion techniques aim to reduce detectability while still gathering useful data — mirroring real-world advanced persistent threat (APT) behavior.

11.2 Common Evasion Techniques

Packet Fragmentation Splitting probe packets into smaller fragments to slip past packet-inspection firewalls that don't properly reassemble fragments before analysis.

```
nmap -f 192.168.1.10
```

Decoy Scanning Sending scan packets that appear to originate from multiple **decoy IP addresses** alongside the real source, making it harder for defenders to identify the actual attacker among the noise.

```
nmap -D RND:10 192.168.1.10
```

Source Port Manipulation Some firewalls trust traffic originating from specific "trusted" source ports (e.g., 53 for DNS, 20 for FTP-data). Spoofing the source port can sometimes bypass these overly permissive rules.

```
nmap --source-port 53 192.168.1.10
```

Timing Templates (Scan Speed Control) Slower scans generate less traffic in a shorter window, making them harder for threshold-based IDS rules to flag.

```
nmap -T0 192.168.1.10 # Paranoid (very slow, highly evasive)
nmap -T5 192.168.1.10 # Insane (very fast, easily detected)
```

(Templates range from `-T0` "paranoid" to `-T5` "insane" — the default is `-T3`)

MAC Address Spoofing On local networks, spoofing the source MAC address can help evade MAC-based access controls or logging.

```
nmap --spoof-mac 0 192.168.1.10
```

Idle (Zombie) Scanning An advanced technique using a third-party "zombie" host with predictable IP ID sequence numbers to scan a target completely blind — the attacker's own IP never directly touches the target, making attribution extremely difficult.

```
nmap -sI zombie_host 192.168.1.10
```

Randomizing Scan Order Scanning hosts or ports in random (rather than sequential) order to avoid triggering pattern-based detection rules.

```
nmap --randomize-hosts 192.168.1.0/24
```

Using Alternative/Uncommon Scan Types FIN, NULL, and Xmas scans (covered in Section 7.2) can sometimes bypass simple stateless packet filters that only track SYN packets.

11.3 Defensive Countermeasures Against Scanning

As a well-rounded student, you should understand both sides:

- **Stateful firewalls** — track connection state, defeating many "half-open" and flag-manipulation scans
- **IDS/IPS tuning** — properly configured systems detect abnormal scan patterns (e.g., many ports touched in a short time from one source)
- **Rate limiting** — throttling connections from a single source to slow down aggressive scans
- **Network segmentation** — limiting what an attacker can reach even after finding a foothold
- **Regular log review and SIEM correlation** — detecting slow, evasive scans that individual alerts might miss requires correlating events over time

12. Summary Workflow: Putting Scanning & Enumeration Together

A professional scanning phase typically follows this sequence:

1. **Confirm scope** — verify exact IP ranges/hosts authorized for active testing
2. **Host discovery** — determine which systems are alive (`nmap -sn`)
3. **Port scanning** — identify open ports on live hosts (`nmap -ss -p-`)
4. **Service/version detection** — identify what's running on each open port (`nmap -sV`)
5. **OS fingerprinting** — determine the underlying operating system (`nmap -O`)
6. **Banner grabbing** — manually confirm/verify version information where needed
7. **NSE scripting** — run targeted scripts for deeper enumeration and vulnerability checks (`nmap --script vuln`)
8. **Apply evasion techniques as needed** — based on scope permissions and stealth requirements
9. **Document everything** — every open port, service, version, and OS finding feeds directly into the next phase: **Vulnerability Analysis**

13. Key Takeaways for Students

- Scanning is the bridge between **passive knowledge** (recon) and **actionable technical detail** (exploitation)
- Master Nmap deeply — it remains the single most important tool in this phase across nearly every real-world engagement and certification exam (CEH, OSCP, Security+, PenTest+)
- Understand the **protocol-level mechanics** (TCP handshake, UDP behavior, ICMP) — memorizing flags without understanding *why* they work will fail you against unusual real-world edge cases
- Always think in terms of **detectability and authorization** — active scanning is inherently visible and must remain strictly within scope
- Practice exclusively on **authorized lab environments** (TryHackMe, HackTheBox, your own local VM lab with tools like Metasploitable) — never against systems you don't own or have explicit written permission to test